

VN30 REAL-TIME ANALYZER

PROJECT & TECHNICAL SPECIFICATION

Hệ thống phân tích kỹ thuật và Machine Learning thời gian thực
cho 30 cổ phiếu thuộc VN30

PHẠM VI CỐT LÕI

Phân tích dữ liệu thị trường -> tính chỉ báo -> tạo feature -> phân loại rủi ro -> sinh Entry/TP/SL -> gửi cảnh báo qua Telegram. Không tự động đặt lệnh.

Ngày lập tài liệu: 24/08/2026
Phiên bản: 1.0 - Design Baseline

1. Tóm tắt dự án

VN30 REAL-TIME ANALYZER là hệ thống phân tích thị trường chạy tự chủ 100% bằng Rust, tập trung vào 30 cổ phiếu trong rổ VN30. Thiết kế ban đầu yêu cầu hệ thống tự động nhận dữ liệu thị trường, duy trì trạng thái thời gian thực, tính các chỉ báo kỹ thuật, xây dựng đặc trưng cho Machine Learning, phân loại rủi ro thành Safe / Medium / Risky và gửi cảnh báo qua Telegram Bot.

Tài liệu này mở rộng báo cáo 1 trang ban đầu thành đặc tả triển khai. Các phần lấy trực tiếp từ file gốc được giữ làm baseline; các phần còn thiếu được bổ sung dưới dạng đề xuất kiến trúc nhằm biến ý tưởng thành một hệ thống có thể phát triển, kiểm thử và vận hành.

NGUYÊN TẮC THIẾT KẾ

Không có auto-execution. Hệ thống chỉ phát hiện, xếp hạng, tính mức rủi ro và cảnh báo; quyết định giao dịch vẫn do người dùng thực hiện.

2. Baseline từ tài liệu gốc

Hạng mục	Yêu cầu đã có trong tài liệu gốc	Trạng thái
Phạm vi	30 cổ phiếu thuộc VN30	Baseline
Mục tiêu	Phân tích kỹ thuật + Machine Learning thời gian thực	Baseline
Phân loại rủi ro	Safe / Medium / Risky	Baseline
Target	Safe +12% đến +15%; Medium +15% đến +18%; Risky +20% đến +25%	Baseline - cần backtest

Stop Loss	Safe -3% đến -4%; Medium -5% đến -6%; Risky -7% đến -8%	Baseline - cần backtest
Beta	Safe <1.0; Medium 1.0-1.25; Risky >1.25	Baseline - cần định nghĩa cách tính
Dynamic Basket	Đồng bộ dữ liệu lúc 08:00 mỗi ngày giao dịch	Giữ lịch vận hành; không coi là thay rõ mỗi ngày
Self-Healing	Reconnect + re-subscribe WebSocket	Baseline
ML	Random Forest, retrain định kỳ Chủ Nhật, hot-swap	Baseline
Alert	Entry / TP / SL + Telegram	Baseline
Tech stack	Tokio, Tokio-Tungstenite, DashMap, Polars, Smartcore, Teloxide	Baseline

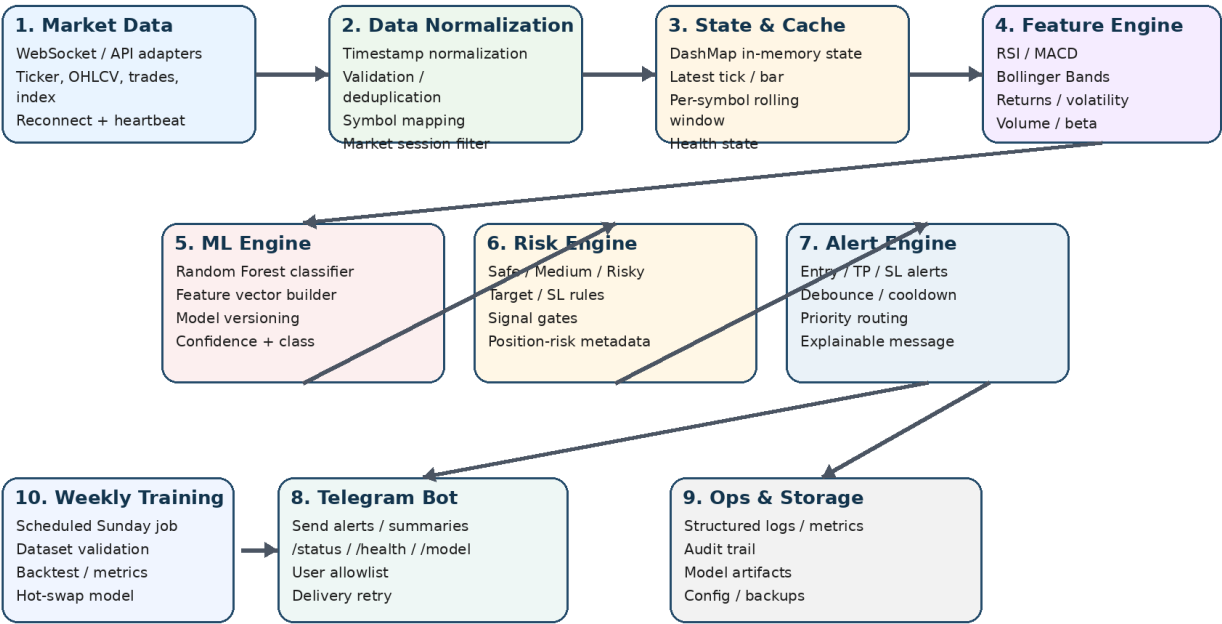
3. Những điểm cần chốt trước khi code

1. Nguồn dữ liệu thị trường cụ thể: tên nhà cung cấp, endpoint, symbol format, rate limit, giấy phép sử dụng và SLA.
2. Định nghĩa “bar”/timeframe dùng cho chỉ báo và model: 1m, 5m, 15m, 1h, ngày hoặc kết hợp.
3. Định nghĩa beta: benchmark, lookback window, frequency, cách xử lý cổ phiếu mới/thiếu dữ liệu.
4. Cách tạo label cho Random Forest và cách bảo đảm không có look-ahead bias hoặc data leakage.
5. Quy tắc chuyển từ risk class sang Target/SL/Entry và thứ tự ưu tiên giữa ML với rule-based risk engine.
6. Quy tắc debounce/cooldown của cảnh báo để tránh spam Telegram.
7. SLA mục tiêu: độ trễ data-to-alert, thời gian reconnect và thời gian khởi động lại.
8. Chính sách lưu trữ dữ liệu, lịch sử tín hiệu, model artifact, log và audit trail.

4. Kiến trúc tổng thể

Kiến trúc đề xuất tách hệ thống thành các module độc lập, giao tiếp bằng các typed event/message. Mục tiêu là dễ kiểm thử, dễ thay nguồn dữ liệu và không buộc toàn bộ ứng dụng phải phụ thuộc vào một vendor cụ thể.

VN30 REAL-TIME ANALYZER - PROPOSED ARCHITECTURE



Hình 1. Kiến trúc đề xuất end-to-end.

5. Cấu trúc module Rust

Module	Trách nhiệm chính	Input	Output
market_data	Kết nối WebSocket/API, heartbeat, reconnect, parse payload	Raw provider messages	NormalizedTick / Trade / Bar
symbol_registry	Quản lý symbol VN30, mapping, trạng thái active	Official basket config	SymbolMeta
state_store	Giữ latest state + rolling windows bằng RAM	Normalized events	LatestState / Window
feature_engine	Tạo OHLCV-derived features + indicators	Windowed bars	FeatureVector
ml_engine	Load/predict/train Random Forest	FeatureVector, label set	RiskClass + score
risk_engine	Rule engine cho target/SL/risk gates	RiskClass, price, beta, volatility	RiskDecision
signal_engine	Sinh Entry/TP/SL event và state transition	RiskDecision + market state	SignalEvent
alert_engine	Debounce, priority, formatting, delivery	SignalEvent	AlertMessage
telegram	Bot commands + outbound messaging	AlertMessage / command	Telegram delivery
scheduler	08:00 sync, Sunday training, periodic maintenance	Cron/time rules	Jobs
storage	Persist snapshots, signals, audits, model metadata	Domain events	DB/files/object store
observability	Structured logs, metrics, health state	All modules	Logs / metrics / health
config	ENV/TOML, validation, runtime toggles	Config files/env	Validated Config

6. Luồng dữ liệu end-to-end

- 9. Data Source gửi tick/quote/trade hoặc bar qua WebSocket/API.
 - 10. Market Data Adapter kiểm tra schema, timestamp, symbol và sequence nếu nhà cung cấp hỗ trợ.
 - 11. Dữ liệu được normalize về một domain model duy nhất.
 - 12. State Store cập nhật latest tick và rolling window cho từng mã.
 - 13. Feature Engine chạy incremental để cập nhật RSI, MACD, Bollinger Bands, returns, volatility, volume và các feature khác.
 - 14. ML Engine nhận feature vector và trả RiskClass + score/confidence.
 - 15. Risk Engine áp rule gating: dữ liệu thiếu, stale data, spread/volatility bất thường, risk policy và Target/SL.
 - 16. Signal Engine quyết định có tạo Entry/TP/SL event hay không.
 - 17. Alert Engine chống spam, gán priority và chuyển message sang Telegram.
 - 18. Observability/Storage ghi lại event, model version, config version và kết quả gửi cảnh báo.
- MarketEvent -> Normalize -> StateStore -> FeatureEngine -> ML -> Risk -> Signal -> Alert -> Telegram

7. Chức năng 1 - Market Data Ingestion

Mục tiêu: tạo một lớp dữ liệu thị trường ổn định, độc lập với vendor. Do tài liệu gốc chưa chỉ định nhà cung cấp dữ liệu, adapter phải được thiết kế theo interface để có thể thay nguồn mà không sửa ML/Risk/Alert.

Chức năng	Mô tả	Mức ưu tiên	Acceptance
WebSocket connect	Kết nối tới endpoint; hỗ trợ TLS khi cần	P0	Kết nối thành công và nhận message
Authentication	Token/key/header theo vendor	P0	Không hard-code secret
Heartbeat	Ping/pong hoặc protocol heartbeat	P0	Phát hiện stale connection
Reconnect	Retry + exponential backoff + jitter	P0	Tự reconnect khi mất kết nối
Re-subscribe	Đăng ký lại symbol sau reconnect	P0	Không mất subscription sau reconnect
Message parsing	Parse raw payload thành enum/domain structs	P0	Malformed message không làm crash process
Deduplication	Loại bản tin trùng theo sequence/timestamp nếu có	P1	Không nhân đôi event
Clock normalization	Chuẩn hóa timestamp về UTC + local display	P0	Không lệch ngày/session
Rate-limit awareness	Theo dõi rate limit / server errors	P1	Không tự retry vô hạn
Data health	Tính age/staleness mỗi symbol	P0	Symbol stale được đánh dấu

8. Chức năng 2 - Dynamic Basket / Symbol Registry

Tài liệu gốc yêu cầu tự động cập nhật danh sách và tỷ trọng lúc 08:00. Về mặt nghiệp vụ, hệ thống nên hiểu đây là “đồng bộ dữ liệu rõ và metadata vận hành”, không phải giả định rằng 30 thành phần được thay đổi mỗi ngày. HOSE công bố VN30 là chỉ số gồm 30 cổ phiếu, tính theo vốn hóa điều chỉnh free-float; tài liệu HOSE-Index v4.0 và factsheet 2026 là nguồn tham chiếu cho rule chính thức.

NciteÖturn267475search24Öturn267475search26Ö

Chức năng	Chi tiết
Basket sync	Tải/cập nhật danh sách thành phần từ nguồn chính thức hoặc file cấu hình đã phê duyệt.
Effective date	Lưu ngày/giờ mà một basket version bắt đầu có hiệu lực.
Symbol mapping	Map ticker vendor -> ticker nội bộ; chuẩn hóa mã.
Metadata	Lưu company name, exchange, active flag và metadata cần cho model.
Validation	Bắt lỗi missing/duplicate/invalid symbols; không kích hoạt basket lỗi.
Fallback	Nếu sync thất bại, giữ basket version cuối cùng hợp lệ và phát cảnh báo vận hành.
Audit	Ghi basket_version, source, fetched_at, checksum và diff với version trước.

9. Chức năng 3 - State Store & Rolling Window

- **Latest tick state:** giá gần nhất, bid/ask nếu có, volume, timestamp, quality flag.
- **Rolling bars:** lưu N bar gần nhất cho từng timeframe để tính indicators.
- **Session state:** pre-open / continuous / break / close / unknown theo lịch thị trường được cấu hình.
- **Staleness state:** fresh / stale / disconnected.
- **Concurrency:** dùng DashMap cho truy cập đồng thời; tránh lock toàn cục khi cập nhật từng symbol.
- **Memory policy:** giới hạn window, eviction và kiểm soát memory footprint.

10. Chức năng 4 - Technical Indicators & Feature Engineering

Polars phù hợp làm lớp DataFrame/query/feature processing; chỉ báo nên được implement thành các hàm domain có test độc lập, thay vì giả định Polars tự cung cấp sẵn toàn bộ technical-analysis API. Tài liệu Polars hiện mô tả đây là DataFrame/analytical query engine trong Rust.

NciteÖturn267475search0Öturn267475search2Ö

Feature group	Ví dụ	Đầu ra nên chuẩn hóa
Trend	EMA/SMA, slope, price vs MA	float + validity
Momentum	RSI, MACD, MACD signal, histogram	float + validity
Volatility	ATR, rolling std, Bollinger width	float + validity
Volume	volume ratio, z-score, turnover proxy	float + validity
Return	1/3/5/10/20-bar returns	float + validity
Market-relative	relative return vs VN30/VNIndex, beta	float + validity
Quality	staleness, missing ratio, outlier flag	enum/bool

11. Chức năng 5 - Beta Engine

Beta là một phần của risk framework gốc, nhưng tài liệu 1 trang chưa định nghĩa phương pháp tính. Cần biến thành configuration để tránh “hard-code” một con số mà không có reproducibility.

Tham số	Đề xuất mặc định ban đầu	Ghi chú
Benchmark	VN30	Có thể đổi sang VNIndex bằng config
Return frequency	Theo timeframe model	Phải đồng nhất với label/model
Lookback	60-120 observations	Chọn bằng backtest; không kết luận trước
Estimator	Cov(asset, benchmark) / Var(benchmark)	Cần xử lý missing data

Minimum samples	Ví dụ 30 observations	Nếu thiếu -> beta unavailable
Outlier handling	Winsorize hoặc robust rule	Phải test impact

QUYẾT ĐỊNH CHƯA KHÓA

Các giá trị lookback/frequency ở trên chỉ là cấu hình đề xuất để bắt đầu backtest, không phải thông số cuối cùng của mô hình.

12. Chức năng 6 - Machine Learning Engine

Tài liệu gốc chỉ định Random Forest. SmartCore hiện có RandomForestClassifier với fit/predict và OOB prediction; phiên bản tài liệu được kiểm tra ngày 24/08/2026 là 0.5.3. Ncitedturn537156search20

Thành phần	Chức năng cần làm
Dataset builder	Tập hợp feature vector theo timestamp và symbol; tạo label theo horizon đã chốt.
Label builder	Xác định Safe/Medium/Risky từ outcome future theo rule đã phê duyệt; tránh leakage.
Split strategy	Time-based split; không random split đơn thuần.
Training	Fit Random Forest; lưu params, dataset hash, code version.
Evaluation	Precision/recall/F1 theo class, confusion matrix, class balance, out-of-time performance.
Calibration	Theo dõi confidence; không coi score model là xác suất nếu chưa calibrate.
Artifact	Lưu model + schema + feature order + training metadata.
Load validation	Kiểm tra model version tương thích feature schema trước khi activate.
Hot swap	Load model mới ở background, validate, rồi atomically đổi active pointer.
Rollback	Khôi phục model version trước nếu health check hoặc metric regression fail.

13. Chức năng 7 - Risk Engine

Risk Engine là lớp deterministic đứng giữa ML và Signal Engine. ML có thể đề xuất RiskClass, nhưng risk policy cuối cùng nên có các hard-gates để hệ thống không tạo tín hiệu khi dữ liệu hoặc trạng thái thị trường không đạt điều kiện.

Risk class	Target baseline	SL baseline	Beta band baseline
Safe	+12% đến +15%	-3% đến -4%	< 1.0
Medium	+15% đến +18%	-5% đến -6%	1.0 đến 1.25
Risky	+20% đến +25%	-7% đến -8%	> 1.25

- Rule precedence phải được định nghĩa: data invalid -> no signal; stale -> no signal; risk gate fail -> no signal; model unavailable -> fallback mode hoặc no signal theo config.
- Target/SL phải được lưu theo signal_id để audit, không chỉ tính động rồi bỏ đi.
- Risk class cần gắn model_version và feature_timestamp để giải thích vì sao tín hiệu được sinh.

14. Chức năng 8 - Entry / TP / SL Signal Engine

Signal	Điều kiện khởi tạo	Thông tin bắt buộc
ENTRY	Rule/ML score vượt ngưỡng + data fresh + risk gate pass	symbol, timestamp, price, risk_class, model_version, score
TAKE_PROFIT	Giá chạm target hoặc đạt condition đã định nghĩa	entry_price, current_price, target, realized_change
STOP_LOSS	Giá chạm SL hoặc invalidation condition	entry_price, current_price, sl, loss
INVALIDATED	Mất điều kiện mô hình/trạng thái	reason, state_before/state_after
RECOVERY	Sau một khoảng thời gian cooldown và điều kiện phục hồi	reason, cooldown_elapsed

15. Chức năng 9 - Alert Engine & Telegram Bot

Teloxide là framework Rust cho Telegram Bot và tài liệu hiện tại nêu hỗ trợ Telegram Bot API 9.1; phiên bản crate được kiểm tra là 0.17.0. NcriteÖturn252376search0Öturn252376search1Ộ

Tính năng	Mô tả
Alert formatting	Template thống nhất: symbol, price, risk, entry, TP, SL, score, reason, timestamp.
Priority	CRITICAL / HIGH / INFO để phân biệt lỗi hệ thống và tín hiệu.
Debounce	Không gửi lặp cùng signal trong khoảng cooldown.
Batch summary	Báo cáo định kỳ cho top signals hoặc health.
Commands	/status, /health, /model, /signals, /help; quyền truy cập theo allowlist.
Retry	Retry transient Telegram errors với backoff; dead-letter nếu thất bại kéo dài.
Delivery audit	Lưu alert_id, telegram result, timestamp, retry_count.
No execution	Không nhận lệnh mua/bán từ Telegram; bot chỉ hiển thị trạng thái/cảnh báo.

16. Chức năng 10 - Scheduler

Job	Lịch	Nhiệm vụ
Basket/config sync	08:00 ngày giao dịch	Kiểm tra basket version, config, metadata
Pre-market health	Trước phiên	Kiểm tra data source, symbols, disk, clock, model
Runtime health	Theo chu kỳ	Staleness, connection, queue depth, CPU/memory
Weekly training	Chủ Nhật	Dataset -> validation -> train -> evaluate -> stage
Model activation	Sau training	Canary/health check -> hot swap -> audit
Backup	Hàng ngày/Hàng tuần	Config, model artifacts, audit metadata
Housekeeping	Theo ngày	Retention logs / snapshots / temp data

17. Chức năng 11 - Self-Healing

Lỗi	Phát hiện	Khôi phục	Escalation
WebSocket disconnected	Socket error / heartbeat timeout	Reconnect + backoff + re-subscribe	Telegram CRITICAL nếu vượt threshold

Stale symbol	now - last_tick > threshold	Mark stale, retry source	Không sinh tín hiệu cho symbol đó
Parser error	Deserialize failure	Drop message + counter	Alert nếu rate tăng
Model load failure	Schema/hash/version mismatch	Giữ model cũ	CRITICAL
Telegram unavailable	HTTP/API error	Retry queue	CRITICAL sau retry budget
Disk/storage full	Metric threshold	Pause non-critical writes / cleanup	CRITICAL
Clock drift	NTP/OS clock check	Resync/flag	Block time-sensitive jobs

18. Chức năng 12 - Storage & Data Lifecycle

Thiết kế baseline nên phân biệt dữ liệu nóng trong RAM với dữ liệu bền vững. DashMap phù hợp cho state thời gian thực; storage bền vững có thể bắt đầu bằng SQLite/PostgreSQL hoặc file Parquet tùy quy mô, nhưng lựa chọn cuối cùng cần phụ thuộc retention và query workload.

Dataset	Mục đích	Retention đề xuất	Độ ưu tiên
Ticks/raw events	Replay/debug/backfill	Configurable	P1
Bars/features	Backtest/training/audit	Months/years	P0
Signals	Audit toàn bộ tín hiệu	Years	P0
Alerts	Audit Telegram delivery	Months/years	P0
Model artifacts	Rollback/reproducibility	Years	P0
Logs/metrics	Observability	Days/weeks	P0
Config/basket versions	Reproducibility	Years	P0

19. Chức năng 13 - Observability & Audit

Metric/Log	Ý nghĩa
data_latency_ms	Latency từ source timestamp đến process
alert_latency_ms	Latency từ event/feature đến alert
websocket_reconnects_total	Số lần reconnect
stale_symbols	Số symbol stale hiện tại
feature_errors_total	Lỗi feature calculation
model_predict_total / error	Theo dõi model runtime
model_version_active	Model đang phục vụ
telegram_send_success / fail	Tình trạng delivery
signal_count_by_class	Phân bố tín hiệu theo risk class
event_loop_lag	Phát hiện nghẽn Tokio
memory_bytes	Theo dõi RAM và rolling-window growth

20. Cấu hình và secrets

```
config.toml
[market_data]
provider = "<TBD>"
reconnect_max_ms = 30000

[model]
artifact = "models/rf_active.bin"
training_schedule = "sunday"

[risk.safe]
target_min = 0.12
target_max = 0.15
```


sl_min = -0.04
sl_max = -0.03

[telegram]
chat_allowlist = ["<TBD>"]

- Secrets không nằm trong Git; dùng environment variables hoặc secret manager.
- Config phải có schema validation trước khi start.
- Mỗi signal phải ghi config_version để tái hiện quyết định.
- Không cho phép runtime command thay đổi risk thresholds nếu chưa có quyền admin/audit.

21. API / Domain Events đề xuất

NormalizedTick { symbol, ts, price, bid, ask, volume, source, quality }
Bar { symbol, timeframe, open, high, low, close, volume, ts }
FeatureVector { symbol, ts, values, schema_version }
Prediction { symbol, ts, class, score, model_version }
RiskDecision { symbol, ts, risk_class, entry, target, stop, reasons }
SignalEvent { signal_id, type, symbol, ts, price, risk, model_version }
AlertMessage { alert_id, signal_id, priority, text, destination }

22. Non-functional requirements

Nhóm	Mục tiêu ban đầu	Cách đo
Latency	Data-to-alert <= 1-2 s cho path bình thường	timestamp event vs alert queue
Availability	>= 99.5% trong giờ vận hành	health monitoring
Reconnect	Tự phục hồi trong threshold cấu hình	disconnect -> healthy
No crash	Một message lỗi không làm process chết	chaos/fuzz tests
Determinism	Cùng input + model/config version -> cùng output	replay test
Auditability	100% signals có model/config/features metadata	DB audit check
Security	Không log secret	secret scanning / log review

LƯU Ý VỀ SỐ LIỆU

Các SLA/threshold ở trên là mục tiêu thiết kế để bắt đầu test, không phải cam kết hiện tại. Cần đo benchmark trên môi trường triển khai thực tế.

23. Testing strategy

Tầng test	Nội dung	Tiêu chí
Unit	Indicators, beta, risk rules, formatting	Coverage + known cases
Property/fuzz	Parser, normalization, edge prices/NaN/missing	Không panic/crash
Integration	WebSocket -> state -> feature -> signal	Đúng event ordering
Replay	Replay historical data qua toàn pipeline	Deterministic + same outputs
Backtest	Đánh giá target/SL/risk class/model	Out-of-time metrics
Chaos	Kill socket, Telegram, storage, model load	Self-healing works
Load	30 symbols * high event rate	No unacceptable lag
Security	Secret leakage, config permissions, bot allowlist	Pass security checklist

24. ML validation và tránh data leakage

- Tất cả feature tại timestamp t chỉ được phép sử dụng dữ liệu có sẵn tại hoặc trước t.
- Label của mẫu t được tạo từ outcome tương lai t+H; không được đưa bất kỳ thông tin hậu kỳ nào vào feature.
- Split train/validation/test theo thời gian; cần một out-of-time holdout.
- Feature schema và feature order phải version hóa.
- Model promotion cần dựa trên test set và baseline; không chỉ dựa trên training score.
- Theo dõi class imbalance và macro metrics; Accuracy đơn thuần có thể gây hiểu nhầm.
- Backtest phải mô phỏng transaction cost/slippage nếu sau này dùng tín hiệu để đánh giá tính thực dụng.

25. Deployment

Thành phần	Khuyến nghị
Runtime	Một binary Rust release; systemd hoặc container.
Config	Mount/read-only config + environment secrets.
Model artifacts	Versioned directory + checksum.
Storage	Local persistent volume hoặc DB server tùy retention.
Log	stdout JSON + file/collector.
Health	/health nếu có HTTP ops endpoint hoặc Telegram /health.
Rollback	Binary + config + model version có thể rollback độc lập.
Time sync	NTP bắt buộc vì hệ thống time-sensitive.

26. Cargo workspace đề xuất

```
vn30-analyzer/  
Cargo.toml  
crates/  
  domain/      # domain structs, enums, validation  
  market-data/ # websocket/api adapters  
  state-store/  # DashMap + rolling windows  
  indicators/   # RSI/MACD/Bollinger/etc.  
  features/     # feature engineering  
  ml/          # SmartCore model training/predict  
  risk/        # deterministic risk rules  
  signals/     # entry/tp/sl state machine  
  alerts/      # debounce + templates  
  telegram/    # Teloxide integration  
  storage/     # DB/files/parquet adapters  
  scheduler/   # scheduled jobs  
  observability/ # tracing + metrics  
  app/         # composition root / binary  
models/  
config/  
tests/  
docs/
```

27. Roadmap triển khai

Phase	Phạm vi	Definition of Done
P0 - Foundation	workspace, config, domain types, logging, health	Binary start ổn định; config validated

P1 - Market data	WebSocket adapter + reconnect + state store	30 symbols live, no crash, stale detection
P2 - Indicators	bars + RSI/MACD/Bollinger + features	Replay tests pass
P3 - Risk engine	Beta + Safe/Medium/Risky + Target/SL rules	Deterministic rules pass
P4 - ML	Dataset + Random Forest + OOT evaluation	Model artifact reproducible
P5 - Signals	Entry/TP/SL state machine	Replay produces expected signals
P6 - Telegram	alerts + commands + debounce	Delivery + retry + allowlist works
P7 - Self-healing/ops	chaos, metrics, backups, rollback	Operational checklist passes
P8 - Production hardening	load test, security, release process	Go-live criteria satisfied

28. Definition of Done - Toàn hệ thống

- Có thể nhận dữ liệu của toàn bộ basket hoạt động mà không mất trạng thái sau reconnect.
- Tất cả event có timestamp, source và quality metadata.
- Indicators/feature engine có unit tests và replay tests.
- Random Forest có dataset version, model artifact, feature schema và out-of-time evaluation.
- Risk Engine tạo quyết định deterministic và audit được.
- Entry/TP/SL có state machine rõ ràng, tránh duplicate signal.
- Telegram chỉ cảnh báo; không có auto-execution.
- Self-healing hoạt động cho WebSocket, model load và Telegram delivery.
- Có metrics, structured logs, model/config version và signal audit trail.
- Có rollback cho binary/config/model và có backup dữ liệu quan trọng.

29. Rủi ro dự án

Rủi ro	Tác động	Giảm thiểu
Nguồn data không ổn định	Mất tín hiệu / stale	Adapter abstraction + retry + health gate
Data quality sai	Model/signal sai	Validation + quality flag + replay
Model leakage	Backtest ảo	Time split + label discipline + OOT
Overfitting	Performance live giảm	Feature restraint + OOT + monitoring
Spam alert	Người dùng bỏ qua cảnh báo	Cooldown + priority + aggregation
Memory growth	Process OOM	Bounded windows + metrics
Vendor/API change	System break	Versioned adapters + contract tests
Config drift	Không tái hiện signal	Config version + immutable audit
SLA không đủ	Alert chậm	Benchmark + profiling + async design

30. Quyết định kiến trúc đề xuất

Quyết định	Lý do
Pure Rust	Giữ đồng nhất runtime; tránh bridge Python trong production path.
Tokio async	Phù hợp nhiều I/O connections và scheduled jobs.
tokio-tungstenite	WebSocket async cho adapter; vendor protocol vẫn tách riêng.
DashMap	State theo symbol có concurrency tốt trong RAM.
Polars	Feature/data transformation và batch processing.
SmartCore	Random Forest thuần Rust theo baseline dự án.
Teloxide	Telegram Bot integration trong Rust.
No auto-execution	Giới hạn phạm vi và rủi ro vận hành theo yêu cầu ban đầu.

31. Tài liệu tham chiếu đã kiểm tra

Các nguồn dưới đây được dùng để kiểm tra các điểm kỹ thuật có thể thay đổi theo phiên bản hoặc theo quy tắc thị trường. Ngày kiểm tra: 24/08/2026.

- HOSE VN30 factsheet (01/2026):
https://staticfile.hsx.vn/Uploads/UploadDocuments/2438018/Form_Factsheet_MCIndices_VN_T02.2026.pdf
- HOSE-INDEX Ground Rules v4.0:
https://staticfile.hsx.vn/Uploads/LocalFiles/ef15ff11e799483abd11677ad0443887/20250114_20241230_QD%20747%20HOSE%20Index%20Ground%20Rules.pdf
- tokio-tungstenite docs (0.30.0): <https://docs.rs/tokio-tungstenite/0.30.0/>
- SmartCore RandomForestClassifier (0.5.3):
https://docs.rs/smartcore/0.5.3/smartcore/ensemble/random_forest_classifier/struct.RandomForestClassifier.html
- Polars docs (0.55.2): <https://docs.rs/polars/0.55.2/>
- Teloxide docs (0.17.0): <https://docs.rs/teloxide/0.17.0/teloxide/>

32. Phụ lục - Danh sách công việc cần thực hiện

- Chốt data vendor và hợp đồng/quyền sử dụng dữ liệu.
- Chốt timeframe, benchmark beta và lookback.
- Chốt định nghĩa label Safe/Medium/Risky.
- Chốt feature list v1 và schema version.
- Implement domain types + event bus/message contracts.
- Implement WebSocket adapter và reconnect.
- Implement basket registry + effective-date + audit.
- Implement rolling window + session state.
- Implement indicators và unit tests.
- Implement beta engine.
- Implement dataset builder + time split.
- Train Random Forest baseline + OOT evaluation.
- Implement risk engine và Target/SL config.
- Implement signal state machine.
- Implement Telegram alerts + commands + allowlist.
- Implement scheduler: 08:00 sync + Sunday training.
- Implement storage + audit trail.
- Implement metrics/logs/health.
- Implement replay test harness.
- Implement chaos/self-healing test.
- Benchmark 30 symbols and define release thresholds.
- Create production runbook + rollback procedure.
- Go-live review: data, model, risk, ops, security.

KẾT LUẬN

Bản đặc tả này đủ để dùng làm baseline cho việc chia task phát triển. Trước khi triển khai production, bốn nhóm

quyết định phải được khóa: data source, timeframes/beta, label & validation của ML, và risk/signal policy. Tất cả ngưỡng hiệu suất trong tài liệu nên được xem là mục tiêu kỹ thuật để benchmark, không phải cam kết lợi nhuận.